Ульяновский государственный университет

Факультет математики, информационных и авиационных технологий

Кафедра Информационных технологий

Лабораторная работа № 1 по теме:

«*Метрическая оценка ПО*».

Выполнил: студент гр. ПРИ-О-25/1
Ермолаев Е.А.

Проверил: Волков М.А.

Ульяновск – 2026 г.

МЕТРИКИ СЛОЖНОСТИ ПРОГРАММЫ.

МЕТРИКИ РАЗМЕРА ПРОГРАММ.

Метрика Холстеда.

Линейная программа.

Учтён исходный текст, включая объявления данных; директивы и комментарии исключены. Строковая константа — один операнд; разделители учтены отдельно. Ключевые слова типов не включены; объявления функций не считаются вызовами.

Словарь операторов	N_i	Словарь операндов	N_i	Словарь операндов	N_i
;	179	std	1	left	14
{	75	Car	10	31	28
[]	8	brand	12	"Brand"	7
}	75	30	12	"Model"	7
(	297	model	12	21	14
)	297	year	13	"Body"	7
do…while	1	price	12	11	14
<<	322	body	12	"Segment"	7
while	23	20	6	8	14
!	16	segment	12	"Year"	7
>>	8	10	6	14	14
||	23	car	76	"Price"	7
<	39	choice	6	116	7
>	35	cout	82	'-'	7
.	153	"1 - Add car"	1	searchChoice	10
clear()	16	endl	70	"search by: "	1
ignore()	22	"2 - Show all"	1	"1 - brand"	1
::	31	"3 - Search"	1	"2 - model"	1
max()	22	"0 - Exit"	1	"3 - year"	1
,	65	cin	78	"4 - price"	1
switch()	1	0	14	"5 - body"	1
case	4	3	3	"6 - segment "	1
:	5	"Enter a number: "	1	"0 - back"	1
getline()	16	numeric_limits	22	6	2
fail()	16	streamsize	22	"Wrong choice. Enter 0-6: "	1
strlen()	8	'\n'	22	searchBrand	5
==	14	1	4	"Enter Brand: "	1

Метрика Холстеда.

Линейная программа — продолжение.

Словарь операторов	N_i	Словарь операндов	N_i	Словарь операндов	N_i
if / if…else	28	"Brand: "	1	"Brand is too long. Maximum 29 characters"	1
<=	3	"Brand is too long. Maximum 29 characters: "	1	nullptr	4
ofstream()	1	"Brand cannot be empty"	2	searchModel	5
|	1	"Model: "	1	"Enter model: "	1
write()	1	"Model is too long. Maximum 29 characters"	1	"Model is too long. Maximum characters 29"	1
reinterpret_cast()	8	"Model cannot be empty"	2	minYear	7
*	8	"Year: "	1	maxYear	6
&	8	1886	3	"From year: "	1
sizeof()	8	2026	3	"To year: "	1
close()	8	"Wrong year. Enter year from 1886 to 2026"	2	"Wrong year. Enter year from "	1
break	5	"Price: "	1	" to 2026: "	1
ifstream()	7	"Wrong price. Enter positive number: "	1	4	1
setw()	84	"Body: "	1	minPrice	5
string()	7	"Body car is too long. Maximum 19 characters"	1	maxPrice	5
read()	7	"Body car cannot be empty"	1	"From price"	1
strstr()	4	"Segment: "	1	"Wrong price. Enter correct price."	2
!=	5	"Segment is too long. Maximum 9 characters"	1	"To price"	1
>=	2	"Segment cannot be empty"	2	5	1
&&	2	file	32	searchBody	5
default	1	"cars.dat"	8	"Enter body: "	1
return	1	ios	9	"Body is too long. Maximum characters 19"	1
		binary	8	"Body cannot be empty"	1
		app	1	searchSegment	5
		"File open error"	7	"Enter segment: "	1
		"Car added."	1	"Segment is too long. Maximum characters 9"	1
		2	2	"Exit"	1
		"Error file read"	1	"Wrong choise"	1

$n_1 = 48$; $n_2 = 108$; $N_1 = 1970$; $N_2 = 850$; $n = 156$; $N = 2820$; $V = N \cdot \log_2 n = 20544.83$.

Метрика Холстеда.

Программа с указателями.

Словарь операторов	N_i	Словарь операндов	N_i	Словарь операндов	N_i
;	180	std	1	left	14
{	75	Car	11	31	28
[]	8	brand	12	"Brand"	7
}	75	30	12	"Model"	7
(	297	model	12	21	14
)	297	year	13	"Body"	7
*	9	price	12	11	14
=	1	body	12	"Segment"	7
new	1	20	6	8	14
do…while	1	segment	12	"Year"	7
<<	322	10	6	14	14
while	23	car	77	"Price"	7
!	16	choice	6	116	7
>>	8	cout	82	'-'	7
||	23	"1 - Add car"	1	searchChoice	10
<	39	endl	70	"search by: "	1
>	35	"2 - Show all"	1	"1 - brand"	1
.	86	"3 - Search"	1	"2 - model"	1
clear()	16	"0 - Exit"	1	"3 - year"	1
ignore()	22	cin	78	"4 - price"	1
::	31	0	14	"5 - body"	1
max()	22	3	3	"6 - segment "	1
,	65	"Enter a number: "	1	"0 - back"	1
switch()	1	numeric_limits	22	6	2
case	4	streamsize	22	"Wrong choice. Enter 0-6: "	1
:	5	'\n'	22	searchBrand	5
getline()	16	1	4	"Enter Brand: "	1

Метрика Холстеда.

Программа с указателями — продолжение.

Словарь операторов	N_i	Словарь операндов	N_i	Словарь операндов	N_i
->	67	"Brand: "	1	"Brand is too long. Maximum 29 characters"	1
fail()	16	"Brand is too long. Maximum 29 characters: "	1	nullptr	4
strlen()	8	"Brand cannot be empty"	2	searchModel	5
==	14	"Model: "	1	"Enter model: "	1
if / if…else	28	"Model is too long. Maximum 29 characters"	1	"Model is too long. Maximum characters 29"	1
<=	3	"Model cannot be empty"	2	minYear	7
ofstream()	1	"Year: "	1	maxYear	6
|	1	1886	3	"From year: "	1
write()	1	2026	3	"To year: "	1
reinterpret_cast()	8	"Wrong year. Enter year from 1886 to 2026"	2	"Wrong year. Enter year from "	1
sizeof()	8	"Price: "	1	" to 2026: "	1
close()	8	"Wrong price. Enter positive number: "	1	4	1
break	5	"Body: "	1	minPrice	5
ifstream()	7	"Body car is too long. Maximum 19 characters"	1	maxPrice	5
setw()	84	"Body car cannot be empty"	1	"From price"	1
string()	7	"Segment: "	1	"Wrong price. Enter correct price."	2
read()	7	"Segment is too long. Maximum 9 characters"	1	"To price"	1
strstr()	4	"Segment cannot be empty"	2	5	1
!=	5	file	32	searchBody	5
>=	2	"cars.dat"	8	"Enter body: "	1
&&	2	ios	9	"Body is too long. Maximum characters 19"	1
default	1	binary	8	"Body cannot be empty"	1
delete	1	app	1	searchSegment	5
return	1	"File open error"	7	"Enter segment: "	1
		"Car added."	1	"Segment is too long. Maximum characters 9"	1
		2	2	"Exit"	1
		"Error file read"	1	"Wrong choise"	1

$n_1 = 51$; $n_2 = 108$; $N_1 = 1967$; $N_2 = 852$; $n = 159$; $N = 2819$; $V = N \cdot \log_2 n = 20615.02$.

Метрика Холстеда.

Программа с модулями.

Словарь операторов	N_i	Словарь операндов	N_i	Словарь операндов	N_i
{	77	Car	18	"Car added."	1
[]	8	brand	6	"Error file read"	1
;	202	30	12	left	2
}	77	model	6	31	4
(	248	year	7	"Brand"	1
)	248	price	6	"Model"	1
do…while	1	body	6	21	2
<<	142	20	6	"Body"	1
while	23	segment	6	11	2
!	16	10	6	"Segment"	1
>>	8	std	4	8	2
||	23	choice	6	"Year"	1
<	39	cout	64	14	2
>	35	"1 - Add car"	1	"Price"	1
.	117	endl	52	116	1
clear()	16	"2 - Show all"	1	'-'	1
ignore()	22	"3 - Search"	1	searchChoice	5
::	31	"0 - Exit"	1	"search by: "	1
max()	22	cin	78	"1 - brand"	1
,	59	0	15	"2 - model"	1
switch()	2	3	3	"3 - year"	1
case	11	"Enter a number: "	1	"4 - price"	1
:	13	numeric_limits	22	"5 - body"	1
addCar()	1	streamsize	22	"6 - segment "	1
break	13	'\n'	22	"0 - back"	1
showAll()	1	1	4	6	2
findCar()	1	2	2	"Wrong choice. Enter 0-6: "	1
default	2	"Exit"	1	4	1
!=	5	"Wrong choice"	1	5	1
return	1	car	55	searchBrand	5

Метрика Холстеда.

Программа с модулями — продолжение.

Словарь операторов	N_i	Словарь операндов	N_i	Словарь операндов	N_i
getline()	16	"Brand: "	1	"Enter Brand: "	1
fail()	16	"Brand is too long. Maximum 29 characters: "	1	"Brand is too long. Maximum 29 characters"	1
strlen()	8	"Brand cannot be empty"	2	nullptr	4
==	8	"Model: "	1	searchModel	5
if / if…else	22	"Model is too long. Maximum 29 characters"	1	"Enter model: "	1
<=	3	"Model cannot be empty"	2	"Model is too long. Maximum characters 29"	1
ofstream()	1	"Year: "	1	minYear	7
|	1	1886	3	maxYear	6
write()	1	2026	3	"From year: "	1
reinterpret_cast()	8	"Wrong year. Enter year from 1886 to 2026"	2	"To year: "	1
*	8	"Price: "	1	"Wrong year. Enter year from "	1
&	9	"Wrong price. Enter positive number: "	1	" to 2026: "	1
sizeof()	8	"Body: "	1	minPrice	5
close()	8	"Body car is too long. Maximum 19 characters"	1	maxPrice	5
ifstream()	7	"Body car cannot be empty"	1	"From price"	1
printHeader()	7	"Segment: "	1	"Wrong price. Enter correct price."	2
read()	7	"Segment is too long. Maximum 9 characters"	1	"To price"	1
printCar()	7	"Segment cannot be empty"	2	searchBody	5
setw()	12	file	32	"Enter body: "	1
string()	1	"cars.dat"	8	"Body is too long. Maximum characters 19"	1
searchByBrand()	1	ios	9	"Body cannot be empty"	1
searchByModel()	1	binary	8	searchSegment	5
searchByYear()	1	app	1	"Enter segment: "	1
searchByPrice()	1	"File open error"	7	"Segment is too long. Maximum characters 9"	1
searchByBody()	1				
searchBySegment()	1				
strstr()	4				
>=	2				
&&	2				

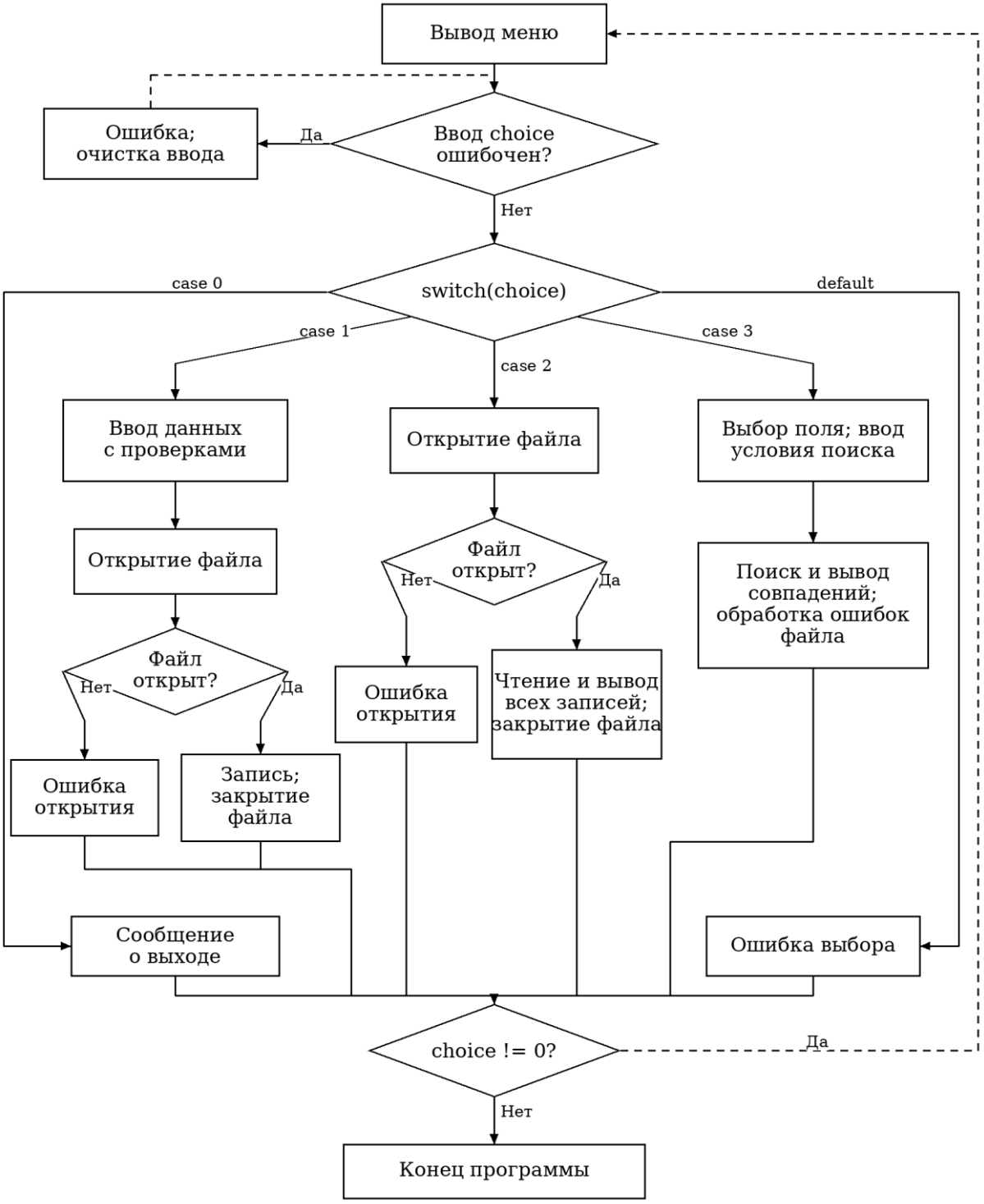
$n_1 = 59$; $n_2 = 108$; $N_1 = 1636$; $N_2 = 632$; $n = 167$; $N = 2268$; $V = N \cdot \log_2 n = 16746.24$.

МЕТРИКИ СЛОЖНОСТИ ПОТОКА УПРАВЛЕНИЯ ПРОГРАММЫ.

Метрика Маккейба.

Линейная программа.

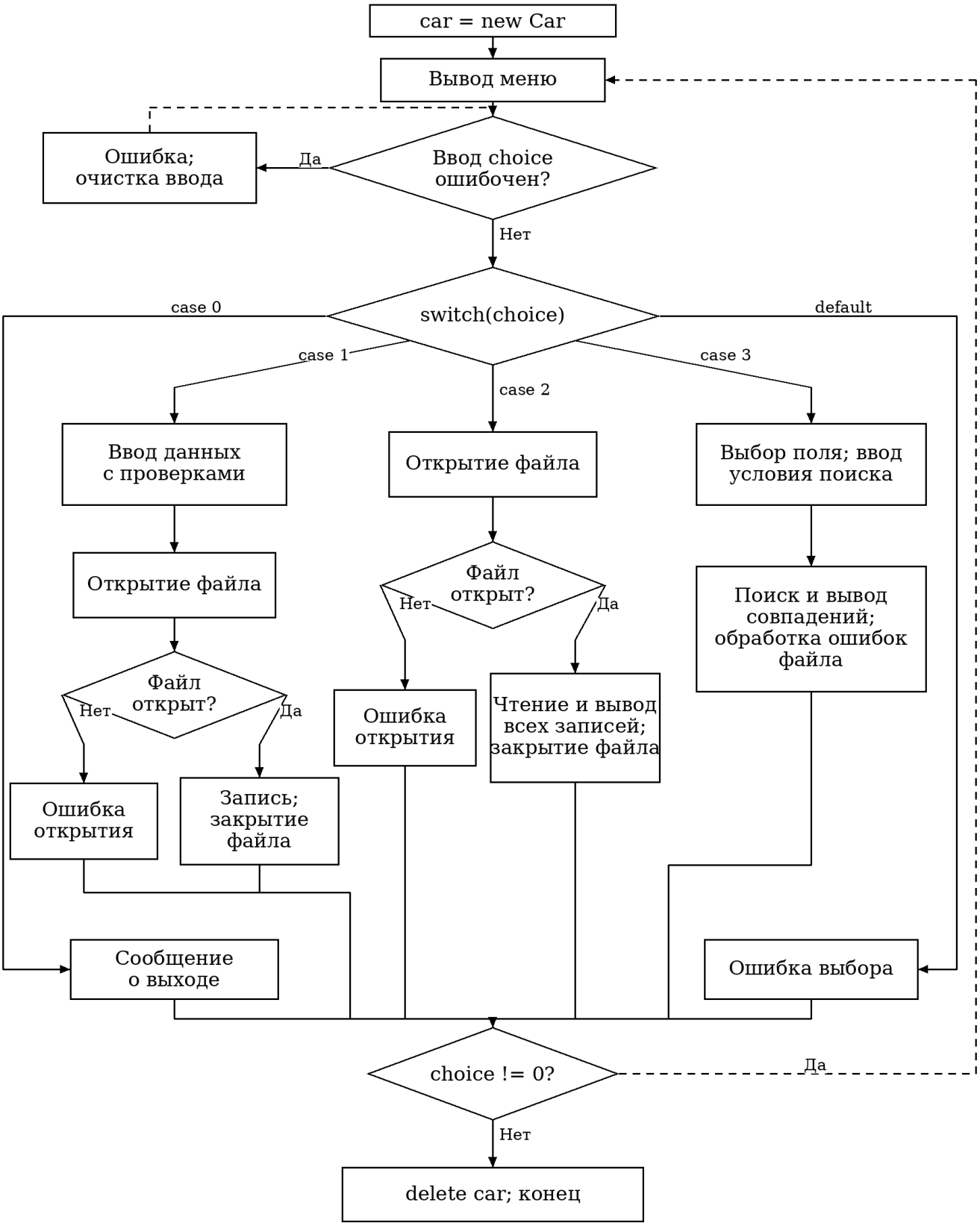
Для представленного графа: G(Z) = 26 − 19 + 2 = 9.



Метрика Маккейба.

Программа с указателями.

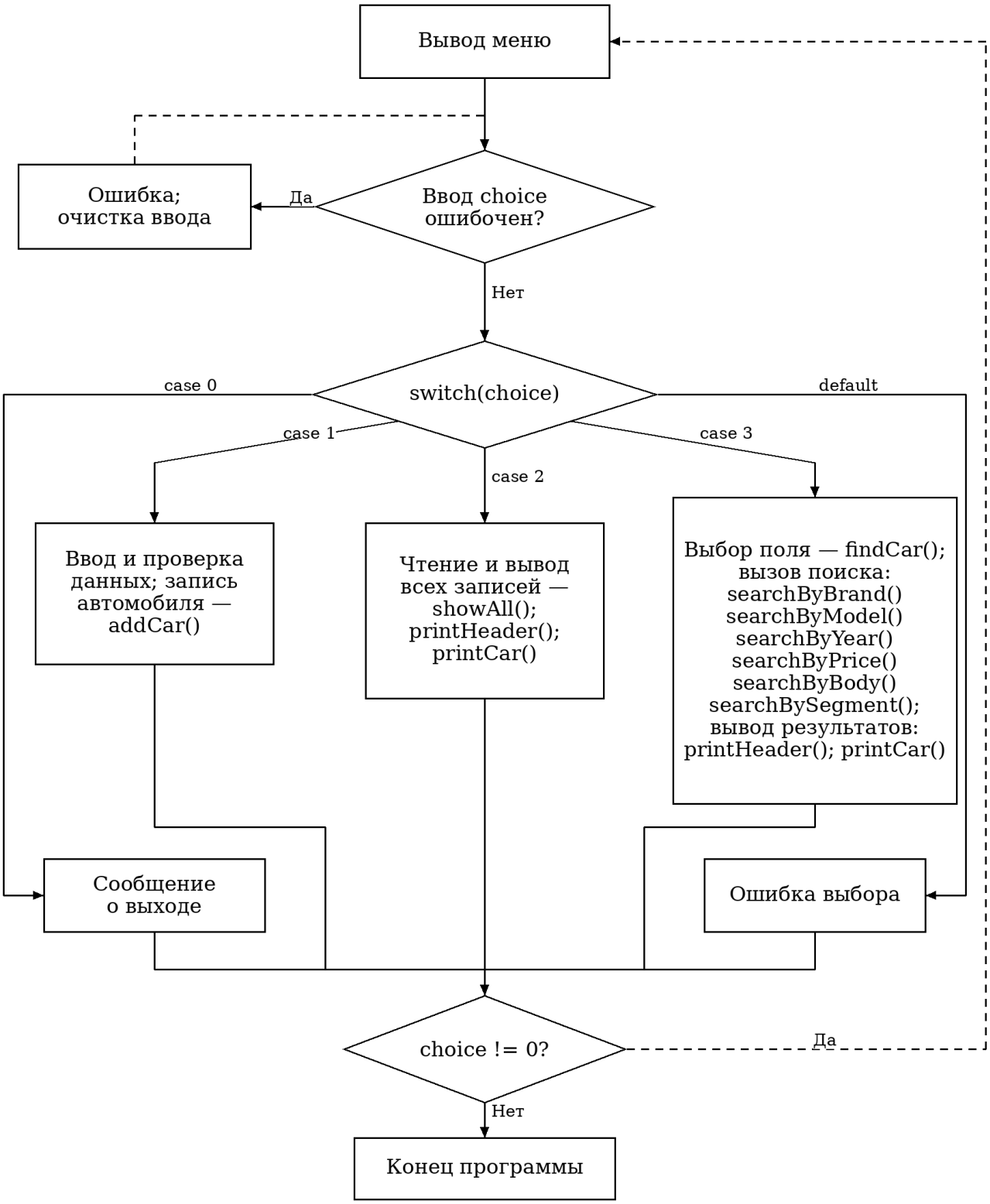
Для представленного графа: G(Z) = 27 − 20 + 2 = 9.



Метрика Маккейба.

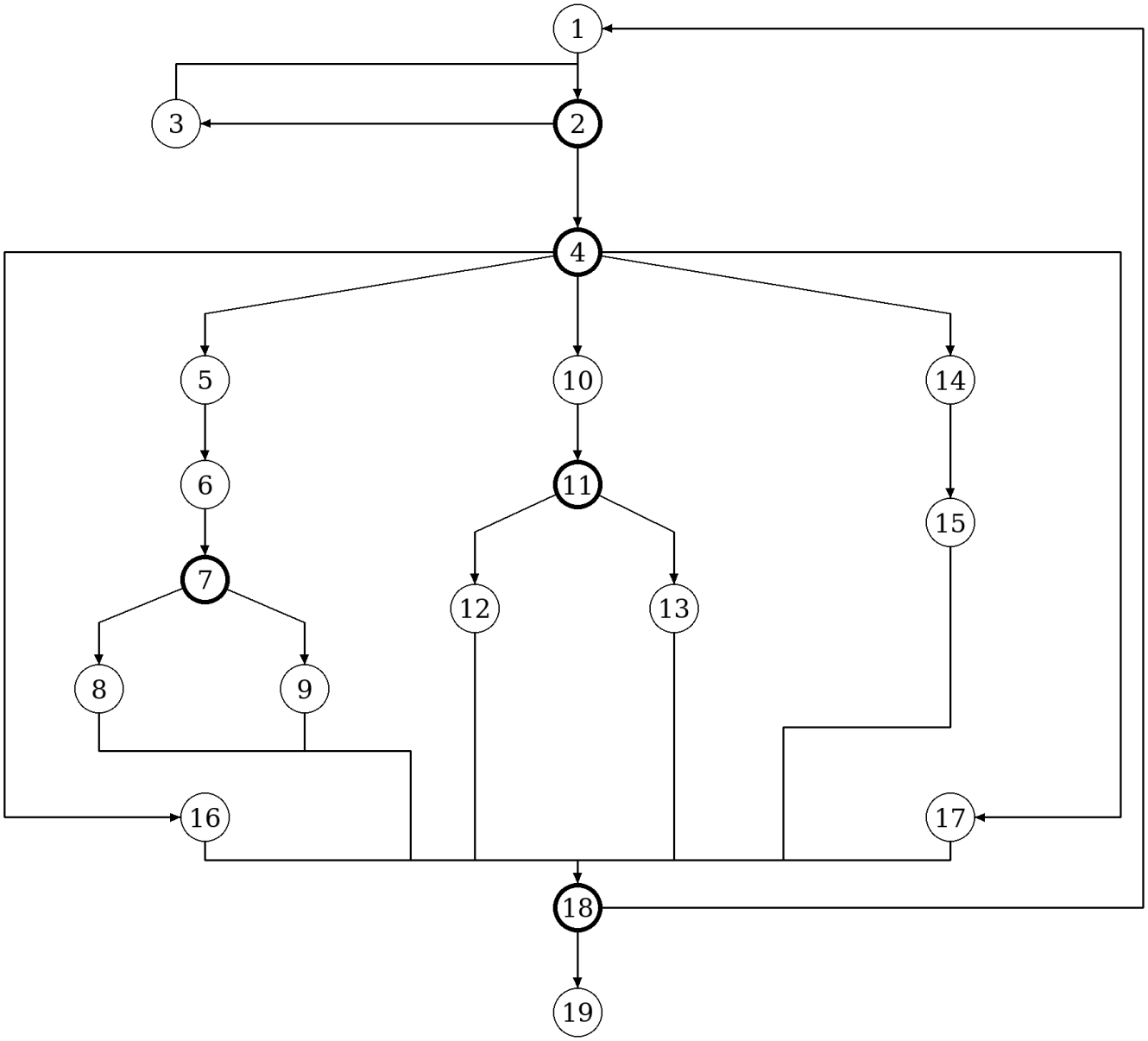
Программа с модулями.

Для представленного графа: G(Z) = 16 − 11 + 2 = 7.



Метрика граничных значений.

Линейная программа.



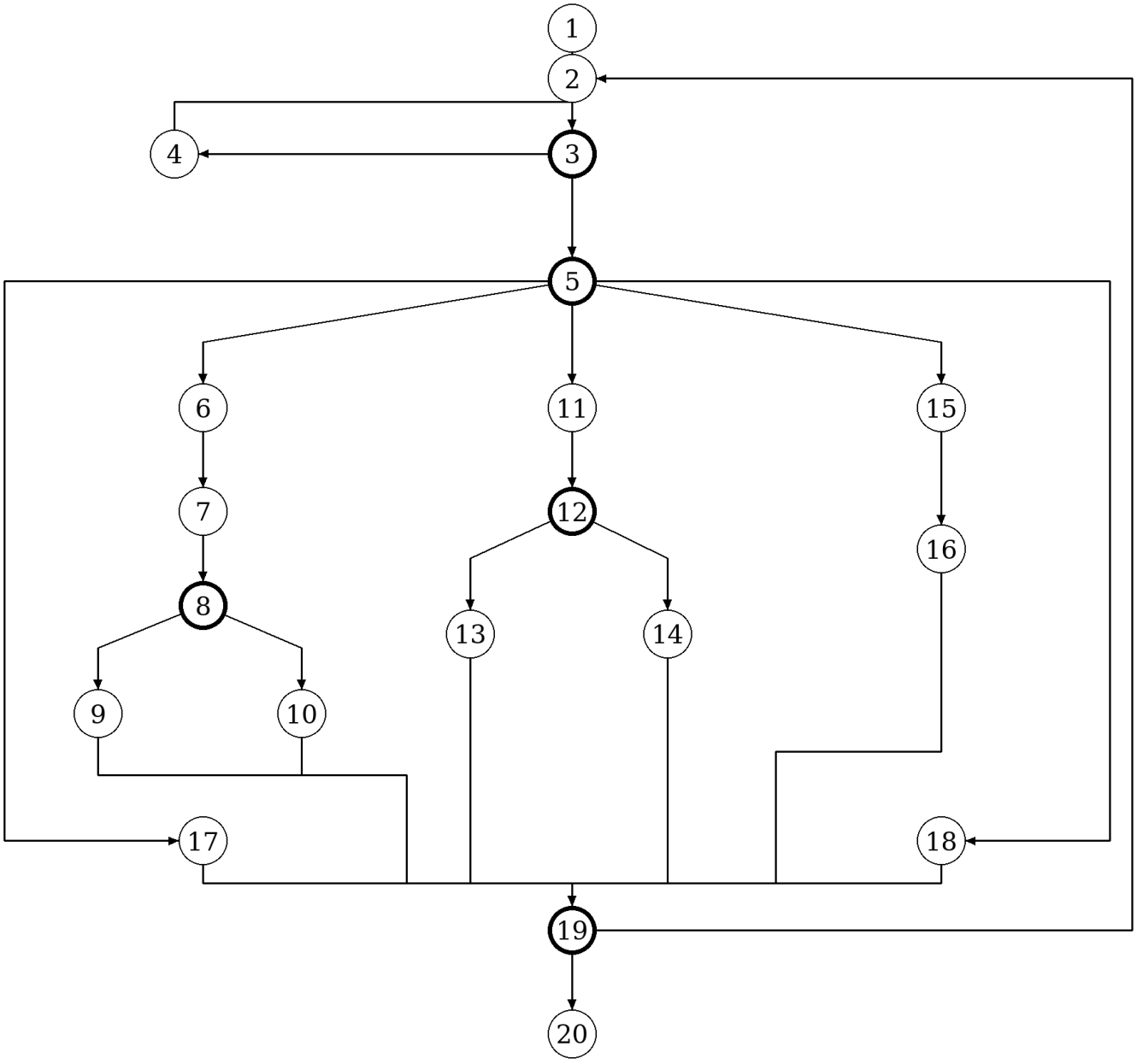
	Вершины отбора				
	2	**4**	**7**	**11**	**18**
Вершины перехода	3, 4	5, 10, 14, 16, 17	8, 9	12, 13	1, 19
Скорректированная сложность вершины	2	14	3	3	18
Вершины подграфа	3	5–17	8, 9	12, 13	1–17
Нижняя граница подграфа	4	18	18	18	19

$$S_{\text{a}} = 2 + 14 + 3 + 3 + 18 + 13 = 53$$

$$S_0 = 1 - \frac{19 - 1}{53} = 1 - \frac{18}{53} \approx 0{,}66$$

Метрика граничных значений.

Программа с указателями.



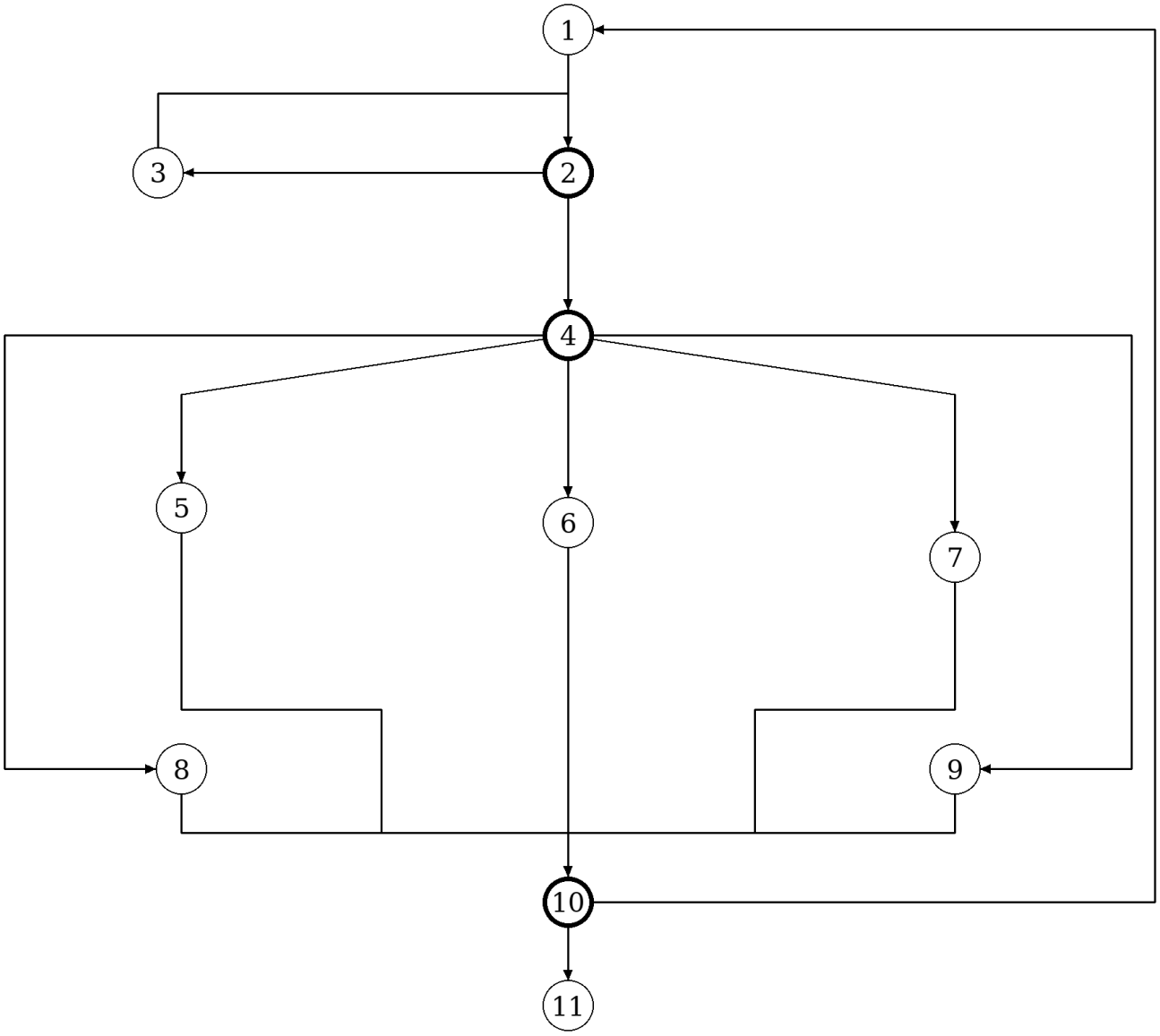
	Вершины отбора				
	3	**5**	**8**	**12**	**19**
Вершины перехода	4, 5	6, 11, 15, 17, 18	9, 10	13, 14	2, 20
Скорректированная сложность вершины	2	14	3	3	18
Вершины подграфа	4	6–18	9, 10	13, 14	2–18
Нижняя граница подграфа	5	19	19	19	20

$$S_{\text{a}} = 2 + 14 + 3 + 3 + 18 + 14 = 54$$

$$S_0 = 1 - \frac{20 - 1}{54} = 1 - \frac{19}{54} \approx 0{,}65$$

Метрика граничных значений.

Программа с модулями.



	Вершины отбора		
	2	**4**	**10**
Вершины перехода	3, 4	5–9	1, 11
Скорректированная сложность вершины	2	6	10
Вершины подграфа	3	5–9	1–9
Нижняя граница подграфа	4	10	11

$$S_a = 2 + 6 + 10 + 7 = 25$$

$$S_0 = 1 - \frac{11 - 1}{25} = 1 - \frac{10}{25} \approx 0{,}60$$

Метрика Джилба.

Линейная программа.

$$CL_1 = N_{if} + N_{switch} = 28 + 1 = 29$$
$$N_1 = 146 + 5 + 1 + 28 + 1 + 23 + 1 = 205$$
$$cl_1 = \frac{CL_1}{N_1} = \frac{29}{205} \approx 0{,}1415$$

Программа с указателями.

$$CL_2 = N_{if} + N_{switch} = 28 + 1 = 29$$
$$N_2 = 147 + 5 + 1 + 28 + 1 + 23 + 1 = 206$$
$$cl_2 = \frac{CL_2}{N_2} = \frac{29}{206} \approx 0{,}1408$$

Программа с модулями.

$$CL_3 = N_{if} + N_{switch} = 22 + 2 = 24$$
$$N_3 = 151 + 13 + 1 + 22 + 2 + 23 + 1 = 213$$
$$cl_3 = \frac{CL_3}{N_3} = \frac{24}{213} \approx 0{,}1127$$

МЕТРИКИ СЛОЖНОСТИ ПОТОКА ДАННЫХ.

Метрика спена.

$$S(v) = n(v) - 1; S_{total} = \sum S(v)$$

Переменная	Линейная	С указателями	С модулями
choice	(5−1) = 4	(5−1) = 4	(5−1) = 4
file	8·(3−1) = 16	8·(3−1) = 16	8·(3−1) = 16
car	(75−1) = 74	(76−1) = 75	(18−1) + (2−1) + (6−1) + 4·(3−1) + 2·(4−1) = 37
searchChoice	(9−1) = 8	(9−1) = 8	(4−1) = 3
searchBrand	(4−1) = 3	(4−1) = 3	(4−1) = 3
searchModel	(4−1) = 3	(4−1) = 3	(4−1) = 3
minYear	(6−1) = 5	(6−1) = 5	(6−1) = 5
maxYear	(5−1) = 4	(5−1) = 4	(5−1) = 4
minPrice	(4−1) = 3	(4−1) = 3	(4−1) = 3
maxPrice	(4−1) = 3	(4−1) = 3	(4−1) = 3
searchBody	(4−1) = 3	(4−1) = 3	(4−1) = 3
searchSegment	(4−1) = 3	(4−1) = 3	(4−1) = 3

$$S_1 = 4 + 16 + 74 + 8 + 3 + 3 + 5 + 4 + 3 + 3 + 3 + 3 = 129$$

$$S_2 = 4 + 16 + 75 + 8 + 3 + 3 + 5 + 4 + 3 + 3 + 3 + 3 = 130$$

$$S_3 = 4 + 16 + 37 + 3 + 3 + 3 + 5 + 4 + 3 + 3 + 3 + 3 = 87$$

Метрика Чепина.

Группа	Линейная	С указателями	С модулями
P	car, searchBrand, searchModel, minYear, maxYear, minPrice, maxPrice, searchBody, searchSegment	car, searchBrand, searchModel, minYear, maxYear, minPrice, maxPrice, searchBody, searchSegment	car (9), searchBrand, searchModel, minYear, maxYear, minPrice, maxPrice, searchBody, searchSegment
M	car, choice, file (8), searchChoice, searchBrand, searchModel, minYear, maxYear, minPrice, maxPrice, searchBody, searchSegment	car, choice, file (8), searchChoice, searchBrand, searchModel, minYear, maxYear, minPrice, maxPrice, searchBody, searchSegment	choice, car (8), file (8), searchChoice, searchBrand, searchModel, minYear, maxYear, minPrice, maxPrice, searchBody, searchSegment
C	car, choice, file (8), searchChoice, searchBrand, searchModel, minYear, maxYear, minPrice, maxPrice, searchBody, searchSegment	car, choice, file (8), searchChoice, searchBrand, searchModel, minYear, maxYear, minPrice, maxPrice, searchBody, searchSegment	choice, file (8), searchChoice, car (6), searchBrand, searchModel, minYear, maxYear, minPrice, maxPrice, searchBody, searchSegment
T	—	—	—

$$Q = P + 2M + 3C + 0{,}5T$$

$$Q_1 = 9 + 2 \cdot 19 + 3 \cdot 19 + 0{,}5 \cdot 0 = 104$$

$$Q_2 = 9 + 2 \cdot 19 + 3 \cdot 19 + 0{,}5 \cdot 0 = 104$$

$$Q_3 = 17 + 2 \cdot 26 + 3 \cdot 24 + 0{,}5 \cdot 0 = 141$$

МЕТРИКИ СТИЛИСТИКИ И ПОНЯТНОСТИ ПРОГРАММ.

Метрика уровня комментированности программ.

Показатель	Линейная	С указателями	С модулями
Всего строк	511	535	555
Строк с комментариями	27	28	45
Строк в сегменте	86 (последний — 81)	90 (последний — 85)	93 (последний — 90)

$$F_i = sign(\frac{N_{\text{комм},i}}{N_i} - 0{,}1); F = \sum_{i=1}^{6} F_i$$

Сегмент	Линейная: комм./стр. → F_i	Указатели: комм./стр. → F_i	Модули: комм./стр. → F_i
1	10/86 → 1	10/90 → 1	11/93 → 1
2	3/86 → -1	3/90 → -1	1/93 → -1
3	3/86 → -1	4/90 → -1	13/93 → 1
4	6/86 → -1	5/90 → -1	5/93 → -1
5	3/86 → -1	3/90 → -1	9/93 → -1
6	2/81 → -1	3/85 → -1	6/90 → -1

$$F_1 = (1) + (-1) + (-1) + (-1) + (-1) + (-1) = -4; K_{\text{комм},1} = \frac{27}{511} \cdot 100\% \approx 5{,}28\%$$

$$F_2 = (1) + (-1) + (-1) + (-1) + (-1) + (-1) = -4; K_{\text{комм},2} = \frac{28}{535} \cdot 100\% \approx 5{,}23\%$$

$$F_3 = (1) + (-1) + (1) + (-1) + (-1) + (-1) = -2; K_{\text{комм},3} = \frac{45}{555} \cdot 100\% \approx 8{,}11\%$$

Метрики Холстеда.

$$n = n_1 + n_2; N = N_1 + N_2; \hat{N} = n_1 log_2 n_1 + n_2 log_2 n_2$$

$$V = N log_2 n; V^* = n log_2 n; L = \frac{V^*}{V}; \hat{L} = \frac{2n_2}{n_1 N_2}$$

$$I = \hat{L} V; E = \frac{V^2}{V^*}$$

Показатель	Линейная	С указателями	С модулями
n_1	48	51	59
n_2	108	108	108
N_1	1970	1967	1636
N_2	850	852	632
n	156	159	167
N	2820	2819	2268
$\hat{N}$	997,6061	1018,8215	1076,6038
V	20544,8343	20615,0171	16746,2413
V*	1136,5227	1162,7484	1233,0786
L	0,0553	0,0564	0,0736
$\hat{L}$	0,0053	0,0050	0,0058
I	108,7668	102,4773	97,0068
E	371387,3885	365495,1765	227427,9961

Метрики Холстеда.

Линейная программа.

$$n = 48 + 108 = 156; N = 1970 + 850 = 2820$$

$$\widehat{N} = 48log_2 48 + 108log_2 108 \approx 997{,}6061$$

$$V = 2820log_2 156 \approx 20544{,}8343$$

$$V^* = 156log_2 156 \approx 1136{,}5227$$

$$L = \frac{1136{,}5227}{20544{,}8343} \approx 0{,}055319$$

$$\hat{L} = \frac{2 \cdot 108}{48 \cdot 850} \approx 0{,}005294$$

$$I = 0{,}005294 \cdot 20544{,}8343 \approx 108{,}7668$$

$$E = \frac{20544{,}8343^2}{1136{,}5227} \approx 371387{,}3885$$

Программа с указателями.

$$n = 51 + 108 = 159; N = 1967 + 852 = 2819$$

$$\widehat{N} = 51log_2 51 + 108log_2 108 \approx 1018{,}8215$$

$$V = 2819log_2 159 \approx 20615{,}0171$$

$$V^* = 159log_2 159 \approx 1162{,}7484$$

$$L = \frac{1162{,}7484}{20615{,}0171} \approx 0{,}056403$$

$$\hat{L} = \frac{2 \cdot 108}{51 \cdot 852} \approx 0{,}004971$$

$$I = 0{,}004971 \cdot 20615{,}0171 \approx 102{,}4773$$

$$E = \frac{20615{,}0171^2}{1162{,}7484} \approx 365495{,}1765$$

Метрики Холстеда.

Программа с модулями.

$$n = 59 + 108 = 167; N = 1636 + 632 = 2268$$

$$\widehat{N} = 59log_2 59 + 108log_2 108 \approx 1076{,}6038$$

$$V = 2268log_2 167 \approx 16746{,}2413$$

$$V^* = 167log_2 167 \approx 1233{,}0786$$

$$L = \frac{1233{,}0786}{16746{,}2413} \approx 0{,}073633$$

$$\hat{L} = \frac{2 \cdot 108}{59 \cdot 632} \approx 0{,}005793$$

$$I = 0{,}005793 \cdot 16746{,}2413 \approx 97{,}0068$$

$$E = \frac{16746{,}2413^2}{1233{,}0786} \approx 227427{,}9961$$

Метрики использования языков программирования и технологических средств.

Метрики Денисенко.

Оценка эффективности контроля соответствия типов данных.

$$g_{\text{тип}} = T\frac{T(T-1) + N_1(n_1-1) + N_2(n_2-1)}{N_1(n_1-T) + N_2(n_2-T)}$$

Показатель	Линейная	С указателями	С модулями
Типы объектов	Car, int, char[], ifstream, ofstream	Car, Car*, int, char[], ifstream, ofstream	Car, const Car&, int, char[], ifstream, ofstream
T	5	6	6
gтип	5,3280	6,4831	6,4513

$$g_{\text{тип},1} = 5\frac{5(5-1) + 1970(48-1) + 850(108-1)}{1970(48-5) + 850(108-5)} \approx 5{,}3280$$

$$g_{\text{тип},2} = 6\frac{6(6-1) + 1967(51-1) + 852(108-1)}{1967(51-6) + 852(108-6)} \approx 6{,}4831$$

$$g_{\text{тип},3} = 6\frac{6(6-1) + 1636(59-1) + 632(108-1)}{1636(59-6) + 632(108-6)} \approx 6{,}4513$$

Оценка эффективности ограничения области доступности переменных.

$$d_M = \frac{T(T-1) + N_1(n_1-1) + N_2(n_2-1) + N_2 n_{\text{св}}}{T(T-1) + N_1(n_1-1) + N_2(n_2-1)}$$

$$d_{M\,1} = \frac{183560 + 850 \cdot 0}{183560} = 1$$

$$d_{M,2} = \frac{189544 + 852 \cdot 0}{189544} = 1$$

$$d_{M,3} = \frac{162542 + 632 \cdot 0}{162542} = 1$$

Уровень языка программирования.

$$\hat{L} = \frac{2n_2}{n_1 N_2}; \lambda = \hat{L}^2 V$$

Показатель	Линейная	С указателями	С модулями
n_1	48	51	59
n_2	108	108	108
N_1	1970	1967	1636
N_2	850	852	632
n	156	159	167
N	2820	2819	2268
V	20544,834257	20615,017051	16746,241335
V*	1136,522746	1162,748390	1233,078617
$\hat{L}$	0,005294	0,004971	0,005793
λ	0,575824	0,509415	0,561936

$$\lambda_1 = (\frac{2 \cdot 108}{48 \cdot 850})^2 \cdot 20544{,}8343 \approx 0{,}575824$$

$$\lambda_2 = (\frac{2 \cdot 108}{51 \cdot 852})^2 \cdot 20615{,}0171 \approx 0{,}509415$$

$$\lambda_3 = (\frac{2 \cdot 108}{59 \cdot 632})^2 \cdot 16746{,}2413 \approx 0{,}561936$$

$$\lambda_{\text{ср}} = \frac{0{,}575824 + 0{,}509415 + 0{,}561936}{3} \approx 0{,}549058$$

Листинги программ.

Линейная программа.

lab1/linear.cpp

```
  1 #include <iostream>
  2 #include <fstream>
  3 #include <cstring>
  4 #include <limits>
  5 #include <iomanip>
  6 #include <string>
  8 using namespace std;
 10 // Структура одной записи каталога автомобилей.
 11 struct Car {
 12   char brand[30]; // Марка: до 29 символов и завершающий нулевой символ.
 13   char model[30]; // Модель: до 29 символов и завершающий нулевой символ.
 14   int year; // Год выпуска автомобиля.
 15   int price; // Цена автомобиля.
 16   char body[20]; // Тип кузова: до 19 символов и завершающий нулевой символ.
 17   char segment[10]; // Сегмент: до 9 символов и завершающий нулевой символ.
 18 };
 20 // Точка входа: повторяет меню до выбора пункта 0.
 21 int main() {
 22   Car car; // Текущая запись автомобиля для ввода, чтения или вывода.
 23   int choice; // Выбранный пункт основного меню: 0 — выход, 1–3 — операции.
 25   do {
 26     cout << "1 - Add car" << endl;
 27     cout << "2 - Show all" << endl;
 28     cout << "3 - Search" << endl;
 29     cout << "0 - Exit" << endl;
 31     while (!(cin >> choice) || choice < 0 || choice > 3) {
 32       cout << "Enter a number: ";
 34       cin.clear();
 35       cin.ignore(numeric_limits<streamsize>::max(), '\n');
 36     }
 37     switch (choice) {
 38     case 1: {
 39       cin.ignore(numeric_limits<streamsize>::max(), '\n');
 41       cout << "Brand: ";
 42       cin.getline(car.brand, 30);
 44       while (cin.fail() || strlen(car.brand) == 0) {
 45         if (cin.fail()) {
 46           cin.clear();
 47           cin.ignore(numeric_limits<streamsize>::max(), '\n');
 49           cout << "Brand is too long. Maximum 29 characters: ";
50-51         } else {
 52           cout << "Brand cannot be empty" << endl;
 53         }
 55         cin.getline(car.brand, 30);
 56       }
 58       cout << "Model: ";
 59       cin.getline(car.model, 30);
 61       while (cin.fail() || strlen(car.model) == 0) {
 62         if (cin.fail()) {
 63           cin.clear();
 64           cin.ignore(numeric_limits<streamsize>::max(), '\n');
 66           cout << "Model is too long. Maximum 29 characters" << endl;
67-68         } else {
 69           cout << "Model cannot be empty" << endl;
 70         }
 72         cin.getline(car.model, 30);
 74       }
 76       cout << "Year: ";
 77       while (!(cin >> car.year) || car.year < 1886 || car.year > 2026) {
 78         cout << "Wrong year. Enter year from 1886 to 2026" << endl;
```

```
 80         cin.clear();
 81         cin.ignore(numeric_limits<streamsize>::max(), '\n');
 82       }
 84       cout << "Price: ";
 85       while (!(cin >> car.price) || car.price <= 0) {
 86         cout << "Wrong price. Enter positive number: ";
 88         cin.clear();
 89         cin.ignore(numeric_limits<streamsize>::max(), '\n');
 90       }
 91       cin.ignore(numeric_limits<streamsize>::max(), '\n');
 93       cout << "Body: ";
 94       cin.getline(car.body, 20);
 96       while (cin.fail() || strlen(car.body) == 0) {
 97         if (cin.fail()) {
 98           cin.clear();
 99           cin.ignore(numeric_limits<streamsize>::max(), '\n');
101           cout << "Body car is too long. Maximum 19 characters" << endl;
102-103         } else {
104           cout << "Body car cannot be empty" << endl;
105         }
106         cin.getline(car.body, 20);
107       }
109       cout << "Segment: ";
110       cin.getline(car.segment, 10);
112       while (cin.fail() || strlen(car.segment) == 0) {
113         if (cin.fail()) {
114           cin.clear();
115           cin.ignore(numeric_limits<streamsize>::max(), '\n');
117           cout << "Segment is too long. Maximum 9 characters" << endl;
118-119         } else {
120           cout << "Segment cannot be empty" << endl;
121         }
122         cin.getline(car.segment, 10);
123       }
125       ofstream file("cars.dat", ios::binary | ios::app); // Поток двоичной записи: новые данные
добавляются в конец файла.
126       if (!file) {
127         cout << "File open error" << endl;
128-129       } else {
130         file.write(reinterpret_cast<char*>(&car), sizeof(Car));
132         file.close();
134         cout << "Car added." << endl;
135       }
137       break;
138     }
139     case 2: {
140       ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
142       if (!file) {
143         cout << "Error file read" << endl;
144-145       } else {
146-153         cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) << "Body"
<< setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
154         cout << string(116, '-') << endl;
155         while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
156-163           cout << left << setw(31) << car.brand << setw(31) << car.model << setw(21) <<
car.body << setw(11) << car.segment << setw(8) << car.year << setw(14) << car.price << endl;
164         }
166         file.close();
167       }
168       break;
169     }
170     case 3: {
171       int searchChoice; // Поле поиска: 1–6; значение 0 возвращает в основное меню.
173       cout << "search by: " << endl;
174       cout << "1 - brand" << endl;
```

```
   175       cout << "2 - model" << endl;
   176       cout << "3 - year" << endl;
   177       cout << "4 - price" << endl;
   178       cout << "5 - body" << endl;
   179       cout << "6 - segment " << endl;
   180       cout << "0 - back" << endl;
   182       while (!(cin >> searchChoice) || searchChoice < 0 || searchChoice > 6) {
   183         cout << "Wrong choice. Enter 0-6: ";
   185         cin.clear();
   186         cin.ignore(numeric_limits<streamsize>::max(), '\n');
   188       }
   190       if (searchChoice == 1) {
   191         char searchBrand[30]; // Подстрока для поиска по марке.
   193         cin.ignore(numeric_limits<streamsize>::max(), '\n');
   195         cout << "Enter Brand: ";
   196         cin.getline(searchBrand, 30);
   198         while (cin.fail() || strlen(searchBrand) == 0) {
   199           if (cin.fail()) {
   200             cin.clear();
   201             cin.ignore(numeric_limits<streamsize>::max(), '\n');
   203             cout << "Brand is too long. Maximum 29 characters" << endl;
204-205           } else {
   206             cout << "Brand cannot be empty" << endl;
   207           }
   208           cin.getline(searchBrand, 30);
   209         }
   211         ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   213         if (!file) {
   214           cout << "File open error" << endl;
215-216         } else {
217-224           cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
   225           cout << string(116, '-') << endl;
   226           while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
   227             if (strstr(car.brand, searchBrand) != nullptr) {
229-236               cout << left << setw(31) << car.brand << setw(31) << car.model << setw(21)
<< car.body << setw(11) << car.segment << setw(8) << car.year << setw(14) << car.price <<
endl;
   237             }
   238           }
   239           file.close();
   240         }
   241       }
   242       if (searchChoice == 2) {
   243         char searchModel[30]; // Подстрока для поиска по модели.
   245         cin.ignore(numeric_limits<streamsize>::max(), '\n');
   247         cout << "Enter model: " << endl;
   249         cin.getline(searchModel, 30);
   250         while (cin.fail() || strlen(searchModel) == 0) {
   251           if (cin.fail()) {
   252             cin.clear();
   253             cin.ignore(numeric_limits<streamsize>::max(), '\n');
   255             cout << "Model is too long. Maximum characters 29" << endl;
256-257           } else {
   258             cout << "Model cannot be empty" << endl;
   259           }
   260           cin.getline(searchModel, 30);
   261         }
   263         ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   265         if (!file) {
   266           cout << "File open error" << endl;
267-268         } else {
269-276           cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
```

```
   277          cout << string(116, '-') << endl;
   278          while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
   279            if (strstr(car.model, searchModel) != nullptr) {
281-288              cout << left << setw(31) << car.brand << setw(31) << car.model << setw(21)
<< car.body << setw(11) << car.segment << setw(8) << car.year << setw(14) << car.price <<
endl;
   289            }
   290          }
   291          file.close();
   292        }
   293      }
   294      if (searchChoice == 3) {
   295        int minYear; // Нижняя граница диапазона годов.
   296        int maxYear; // Верхняя граница диапазона годов; не меньше нижней.
   297        cout << "From year: " << endl;
   298        while (!(cin >> minYear) || minYear < 1886 || minYear > 2026) {
   299          cout << "Wrong year. Enter year from 1886 to 2026" << endl;
   301          cin.clear();
   302          cin.ignore(numeric_limits<streamsize>::max(), '\n');
   303        }
   304        cout << "To year: " << endl;
   305        while (!(cin >> maxYear) || maxYear < 1886 || maxYear > 2026 || maxYear < minYear)
{
   306          cout << "Wrong year. Enter year from " << minYear << " to 2026: ";
   308          cin.clear();
   309          cin.ignore(numeric_limits<streamsize>::max(), '\n');
   310        }
   312        ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   313        if (!file) {
   314          cout << "File open error" << endl;
315-316        } else {
317-324          cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
   325          cout << string(116, '-') << endl;
   326          while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
   327            if (car.year >= minYear && car.year <= maxYear) {
329-336              cout << left << setw(31) << car.brand << setw(31) << car.model << setw(21)
<< car.body << setw(11) << car.segment << setw(8) << car.year << setw(14) << car.price <<
endl;
   337            }
   338          }
   339          file.close();
   340        }
   341      }
   342      if (searchChoice == 4) {
   343        int minPrice; // Нижняя граница диапазона цен.
   344        int maxPrice; // Верхняя граница диапазона цен; не меньше нижней.
   345        cout << "From price" << endl;
   346        while (!(cin >> minPrice) || minPrice < 1) {
   347          cout << "Wrong price. Enter correct price." << endl;
   349          cin.clear();
   350          cin.ignore(numeric_limits<streamsize>::max(), '\n');
   351        }
   352        cout << "To price" << endl;
   353        while (!(cin >> maxPrice) || maxPrice < 1 || maxPrice < minPrice) {
   354          cout << "Wrong price. Enter correct price." << endl;
   356          cin.clear();
   357          cin.ignore(numeric_limits<streamsize>::max(), '\n');
   358        }
   360        ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   362        if (!file) {
   363          cout << "File open error" << endl;
364-365        } else {
```

```
366-373          cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
   374          cout << string(116, '-') << endl;
   375          while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
   376            if (car.price >= minPrice && car.price <= maxPrice) {
378-385              cout << left << setw(31) << car.brand << setw(31) << car.model << setw(21)
<< car.body << setw(11) << car.segment << setw(8) << car.year << setw(14) << car.price <<
endl;
   386            }
   387          }
   388          file.close();
   389        }
   390      }
   391      if (searchChoice == 5) {
   392        char searchBody[20]; // Подстрока для поиска по типу кузова.
   394        cin.ignore(numeric_limits<streamsize>::max(), '\n');
   396        cout << "Enter body: " << endl;
   397        cin.getline(searchBody, 20);
   398        while (cin.fail() || strlen(searchBody) == 0) {
   399          if (cin.fail()) {
   400            cin.clear();
   401            cin.ignore(numeric_limits<streamsize>::max(), '\n');
   403            cout << "Body is too long. Maximum characters 19" << endl;
404-405           } else {
   406            cout << "Body cannot be empty" << endl;
   407          }
   408          cin.getline(searchBody, 20);
   409        }
   411        ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   413        if (!file) {
   414          cout << "File open error" << endl;
415-416         } else {
417-424           cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
   425          cout << string(116, '-') << endl;
   427          while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
   428            if (strstr(car.body, searchBody) != nullptr) {
430-437              cout << left << setw(31) << car.brand << setw(31) << car.model << setw(21)
<< car.body << setw(11) << car.segment << setw(8) << car.year << setw(14) << car.price <<
endl;
   438            }
   439          }
   440          file.close();
   441        }
   442      }
   443      if (searchChoice == 6) {
   444        char searchSegment[10]; // Подстрока для поиска по сегменту.
   446        cin.ignore(numeric_limits<streamsize>::max(), '\n');
   448        cout << "Enter segment: " << endl;
   449        cin.getline(searchSegment, 10);
   451        while (cin.fail() || strlen(searchSegment) == 0) {
   452          if (cin.fail()) {
   453            cin.clear();
   454            cin.ignore(numeric_limits<streamsize>::max(), '\n');
   456            cout << "Segment is too long. Maximum characters 9" << endl;
457-458           } else {
   459            cout << "Segment cannot be empty" << endl;
   460          }
   461          cin.getline(searchSegment, 10);
   462        }
   464        ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   466        if (!file) {
   467          cout << "File open error" << endl;
468-469         } else {
```

```
470-477           cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
   478           cout << string(116, '-') << endl;
   479           while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
   480             if (strstr(car.segment, searchSegment) != nullptr) {
481-488               cout << left << setw(31) << car.brand << setw(31) << car.model << setw(21)
<< car.body << setw(11) << car.segment << setw(8) << car.year << setw(14) << car.price <<
endl;
   489             }
   490           }
   491           file.close();
   492         }
   494       }
   495       break;
   496     }
   497     case 0:
   498     {
   499       cout << "Exit" << endl;
   500       break;
   501     }
   503     default:
   504       cout << "Wrong choise" << endl;
   505       break;
   506     }
   508   } while (choice != 0);
   510   return 0;
   511 }
```

Программа с указателями.

lab1/pointers.cpp

```
 1  #include <iostream>
 2  #include <fstream>
 3  #include <cstring>
 4  #include <limits>
 5  #include <iomanip>
 6  #include <string>
 8  using namespace std;
10  // Структура одной записи каталога автомобилей.
11  struct Car {
12    char brand[30]; // Марка: до 29 символов и завершающий нулевой символ.
13    char model[30]; // Модель: до 29 символов и завершающий нулевой символ.
14    int year; // Год выпуска автомобиля.
15    int price; // Цена автомобиля.
16    char body[20]; // Тип кузова: до 19 символов и завершающий нулевой символ.
17    char segment[10]; // Сегмент: до 9 символов и завершающий нулевой символ.
18  };
20  // Точка входа: повторяет меню до выбора пункта 0.
21  int main() {
22    Car* car = new Car; // Указатель на запись автомобиля, созданную в динамической
памяти.
23    int choice; // Выбранный пункт основного меню: 0 — выход, 1–3 — операции.
25    do {
26      cout << "1 - Add car" << endl;
27      cout << "2 - Show all" << endl;
28      cout << "3 - Search" << endl;
29      cout << "0 - Exit" << endl;
31      while (!(cin >> choice) || choice < 0 || choice > 3) {
32        cout << "Enter a number: ";
34        cin.clear();
35        cin.ignore(numeric_limits<streamsize>::max(), '\n');
36      }
38      switch (choice) {
39      case 1: {
40        cin.ignore(numeric_limits<streamsize>::max(), '\n');
42        cout << "Brand: ";
43        cin.getline(car->brand, 30);
45        while (cin.fail() || strlen(car->brand) == 0) {
46          if (cin.fail()) {
47            cin.clear();
48            cin.ignore(numeric_limits<streamsize>::max(), '\n');
50            cout << "Brand is too long. Maximum 29 characters: ";
51          } else {
52            cout << "Brand cannot be empty" << endl;
53          }
55          cin.getline(car->brand, 30);
56        }
58        cout << "Model: ";
59        cin.getline(car->model, 30);
61        while (cin.fail() || strlen(car->model) == 0) {
62          if (cin.fail()) {
63            cin.clear();
64            cin.ignore(numeric_limits<streamsize>::max(), '\n');
66            cout << "Model is too long. Maximum 29 characters" << endl;
67          } else {
68            cout << "Model cannot be empty" << endl;
69          }
71          cin.getline(car->model, 30);
72        }
74        cout << "Year: ";
75        while (!(cin >> car->year) || car->year < 1886 || car->year > 2026) {
76          cout << "Wrong year. Enter year from 1886 to 2026" << endl;
78          cin.clear();
79          cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```
    80        }
    82        cout << "Price: ";
    83        while (!(cin >> car->price) || car->price <= 0) {
    84          cout << "Wrong price. Enter positive number: ";
    86          cin.clear();
    87          cin.ignore(numeric_limits<streamsize>::max(), '\n');
    88        }
    90        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    92        cout << "Body: ";
    93        cin.getline(car->body, 20);
    95        while (cin.fail() || strlen(car->body) == 0) {
    96          if (cin.fail()) {
    97            cin.clear();
    98            cin.ignore(numeric_limits<streamsize>::max(), '\n');
   100            cout << "Body car is too long. Maximum 19 characters" << endl;
   101          } else {
   102            cout << "Body car cannot be empty" << endl;
   103          }
   105          cin.getline(car->body, 20);
   106        }
   108        cout << "Segment: ";
   109        cin.getline(car->segment, 10);
   111        while (cin.fail() || strlen(car->segment) == 0) {
   112          if (cin.fail()) {
   113            cin.clear();
   114            cin.ignore(numeric_limits<streamsize>::max(), '\n');
   116            cout << "Segment is too long. Maximum 9 characters" << endl;
   117          } else {
   118            cout << "Segment cannot be empty" << endl;
   119          }
   121          cin.getline(car->segment, 10);
   122        }
   124        ofstream file("cars.dat", ios::binary | ios::app); // Поток двоичной записи: новые данные
добавляются в конец файла.
   126        if (!file) {
   127          cout << "File open error" << endl;
   128        } else {
   129          file.write(reinterpret_cast<char*>(car), sizeof(Car));
   130          file.close();
   132          cout << "Car added." << endl;
   133        }
   135        break;
   136      }
   138      case 2: {
   139        ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   141        if (!file) {
   142          cout << "Error file read" << endl;
   143        } else {
144-151          cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) << "Body"
<< setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
   153          cout << string(116, '-') << endl;
   155          while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
156-163            cout << left << setw(31) << car->brand << setw(31) << car->model << setw(21)
<< car->body << setw(11) << car->segment << setw(8) << car->year << setw(14) << car->price
<< endl;
   164          }
   166          file.close();
   167        }
   169        break;
   170      }
   172      case 3: {
   173        int searchChoice; // Поле поиска: 1–6; значение 0 возвращает в основное меню.
   175        cout << "search by: " << endl;
   176        cout << "1 - brand" << endl;
   177        cout << "2 - model" << endl;
```

```
   178       cout << "3 - year" << endl;
   179       cout << "4 - price" << endl;
   180       cout << "5 - body" << endl;
   181       cout << "6 - segment " << endl;
   182       cout << "0 - back" << endl;
   184       while (!(cin >> searchChoice) || searchChoice < 0 || searchChoice > 6) {
   185         cout << "Wrong choice. Enter 0-6: ";
   187         cin.clear();
   188         cin.ignore(numeric_limits<streamsize>::max(), '\n');
   189       }
   191       if (searchChoice == 1) {
   192         char searchBrand[30]; // Подстрока для поиска по марке.
   194         cin.ignore(numeric_limits<streamsize>::max(), '\n');
   196         cout << "Enter Brand: ";
   197         cin.getline(searchBrand, 30);
   199         while (cin.fail() || strlen(searchBrand) == 0) {
   200           if (cin.fail()) {
   201             cin.clear();
   202             cin.ignore(numeric_limits<streamsize>::max(), '\n');
   204             cout << "Brand is too long. Maximum 29 characters" << endl;
   205           } else {
   206             cout << "Brand cannot be empty" << endl;
   207           }
   209           cin.getline(searchBrand, 30);
   210         }
   212         ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   214         if (!file) {
   215           cout << "File open error" << endl;
   216         } else {
217-224           cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
   226           cout << string(116, '-') << endl;
   228           while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
   229             if (strstr(car->brand, searchBrand) != nullptr) {
230-237               cout << left << setw(31) << car->brand << setw(31) << car->model <<
setw(21) << car->body << setw(11) << car->segment << setw(8) << car->year << setw(14) <<
car->price << endl;
   238             }
   239           }
   241           file.close();
   242         }
   243       }
   245       if (searchChoice == 2) {
   246         char searchModel[30]; // Подстрока для поиска по модели.
   248         cin.ignore(numeric_limits<streamsize>::max(), '\n');
   250         cout << "Enter model: " << endl;
   251         cin.getline(searchModel, 30);
   253         while (cin.fail() || strlen(searchModel) == 0) {
   254           if (cin.fail()) {
   255             cin.clear();
   256             cin.ignore(numeric_limits<streamsize>::max(), '\n');
   258             cout << "Model is too long. Maximum characters 29" << endl;
   259           } else {
   260             cout << "Model cannot be empty" << endl;
   261           }
   263           cin.getline(searchModel, 30);
   264         }
   266         ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   268         if (!file) {
   269           cout << "File open error" << endl;
   270         } else {
271-278           cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
   280           cout << string(116, '-') << endl;
```

```
   282           while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
   283             if (strstr(car->model, searchModel) != nullptr) {
284-291               cout << left << setw(31) << car->brand << setw(31) << car->model <<
setw(21) << car->body << setw(11) << car->segment << setw(8) << car->year << setw(14) <<
car->price << endl;
   292             }
   293           }
   295           file.close();
   296         }
   297       }
   299       if (searchChoice == 3) {
   300         int minYear; // Нижняя граница диапазона годов.
   301         int maxYear; // Верхняя граница диапазона годов; не меньше нижней.
   303         cout << "From year: " << endl;
   305         while (!(cin >> minYear) || minYear < 1886 || minYear > 2026) {
   306           cout << "Wrong year. Enter year from 1886 to 2026" << endl;
   308           cin.clear();
   309           cin.ignore(numeric_limits<streamsize>::max(), '\n');
   310         }
   312         cout << "To year: " << endl;
314-315           while (!(cin >> maxYear) || maxYear < 1886 || maxYear > 2026 || maxYear <
minYear) {
   316           cout << "Wrong year. Enter year from " << minYear << " to 2026: ";
   318           cin.clear();
   319           cin.ignore(numeric_limits<streamsize>::max(), '\n');
   320         }
   322         ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   324         if (!file) {
   325           cout << "File open error" << endl;
   326         } else {
327-334           cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
   336           cout << string(116, '-') << endl;
   338           while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
   339             if (car->year >= minYear && car->year <= maxYear) {
340-347               cout << left << setw(31) << car->brand << setw(31) << car->model <<
setw(21) << car->body << setw(11) << car->segment << setw(8) << car->year << setw(14) <<
car->price << endl;
   348             }
   349           }
   351           file.close();
   352         }
   353       }
   355       if (searchChoice == 4) {
   356         int minPrice; // Нижняя граница диапазона цен.
   357         int maxPrice; // Верхняя граница диапазона цен; не меньше нижней.
   359         cout << "From price" << endl;
   361         while (!(cin >> minPrice) || minPrice < 1) {
   362           cout << "Wrong price. Enter correct price." << endl;
   364           cin.clear();
   365           cin.ignore(numeric_limits<streamsize>::max(), '\n');
   366         }
   368         cout << "To price" << endl;
   370         while (!(cin >> maxPrice) || maxPrice < 1 || maxPrice < minPrice) {
   371           cout << "Wrong price. Enter correct price." << endl;
   373           cin.clear();
   374           cin.ignore(numeric_limits<streamsize>::max(), '\n');
   375         }
   377         ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   379         if (!file) {
   380           cout << "File open error" << endl;
   381         } else {
382-389           cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
```

```
   391          cout << string(116, '-') << endl;
   393          while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
   394            if (car->price >= minPrice && car->price <= maxPrice) {
395-402              cout << left << setw(31) << car->brand << setw(31) << car->model <<
setw(21) << car->body << setw(11) << car->segment << setw(8) << car->year << setw(14) <<
car->price << endl;
   403            }
   404          }
   406          file.close();
   407        }
   408      }
   410      if (searchChoice == 5) {
   411        char searchBody[20]; // Подстрока для поиска по типу кузова.
   413        cin.ignore(numeric_limits<streamsize>::max(), '\n');
   415        cout << "Enter body: " << endl;
   416        cin.getline(searchBody, 20);
   418        while (cin.fail() || strlen(searchBody) == 0) {
   419          if (cin.fail()) {
   420            cin.clear();
   421            cin.ignore(numeric_limits<streamsize>::max(), '\n');
   423            cout << "Body is too long. Maximum characters 19" << endl;
   424          } else {
   425            cout << "Body cannot be empty" << endl;
   426          }
   428          cin.getline(searchBody, 20);
   429        }
   431        ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   433        if (!file) {
   434          cout << "File open error" << endl;
   435        } else {
436-443          cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
   445          cout << string(116, '-') << endl;
   447          while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
   448            if (strstr(car->body, searchBody) != nullptr) {
449-456              cout << left << setw(31) << car->brand << setw(31) << car->model <<
setw(21) << car->body << setw(11) << car->segment << setw(8) << car->year << setw(14) <<
car->price << endl;
   457            }
   458          }
   460          file.close();
   461        }
   462      }
   464      if (searchChoice == 6) {
   465        char searchSegment[10]; // Подстрока для поиска по сегменту.
   467        cin.ignore(numeric_limits<streamsize>::max(), '\n');
   469        cout << "Enter segment: " << endl;
   470        cin.getline(searchSegment, 10);
   472        while (cin.fail() || strlen(searchSegment) == 0) {
   473          if (cin.fail()) {
   474            cin.clear();
   475            cin.ignore(numeric_limits<streamsize>::max(), '\n');
   477            cout << "Segment is too long. Maximum characters 9" << endl;
   478          } else {
   479            cout << "Segment cannot be empty" << endl;
   480          }
   482          cin.getline(searchSegment, 10);
   483        }
   485        ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла
cars.dat.
   487        if (!file) {
   488          cout << "File open error" << endl;
   489        } else {
490-497          cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) <<
"Body" << setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
```

```
   499           cout << string(116, '-') << endl;
   501           while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
   502             if (strstr(car->segment, searchSegment) != nullptr) {
503-510               cout << left << setw(31) << car->brand << setw(31) << car->model <<
setw(21) << car->body << setw(11) << car->segment << setw(8) << car->year << setw(14) <<
car->price << endl;
   511             }
   512           }
   514           file.close();
   515         }
   516       }
   518       break;
   519     }
   521     case 0: {
   522       cout << "Exit" << endl;
   523       break;
   524     }
   526     default:
   527       cout << "Wrong choise" << endl;
   528       break;
   529     }
   531   } while (choice != 0);
   533   delete car; // Освобождение памяти, выделенной для записи через new.
   534   return 0;
   535 }
```

Программа с модулями.

modular/car.h

```
#ifndef CAR_H
#define CAR_H
// Структура одной записи каталога автомобилей.
struct Car {
  char brand[30]; // Марка: до 29 символов и завершающий нулевой символ.
  char model[30]; // Модель: до 29 символов и завершающий нулевой символ.
  int year; // Год выпуска автомобиля.
  int price; // Цена автомобиля.
   char body[20]; // Тип кузова: до 19 символов и завершающий нулевой символ.
   char segment[10]; // Сегмент: до 9 символов и завершающий нулевой символ.
};
#endif
```

modular/main.cpp

```
#include <iostream>
#include <limits>
#include "input.h"
#include "output.h"
#include "search.h"
using namespace std;
// Точка входа: повторяет меню до выбора пункта 0.
int main(){
  int choice; // Выбранный пункт основного меню: 0 — выход, 1–3 — операции.
  do {
    cout << "1 - Add car" << endl;
    cout << "2 - Show all" << endl;
    cout << "3 - Search" << endl;
    cout << "0 - Exit" << endl;
    while (!(cin >> choice) || choice < 0 || choice > 3) {
      cout << "Enter a number: ";
      cin.clear();
      cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    switch (choice) {
      case 1:
      addCar();
        break;
      case 2:
      showAll();
        break;
      case 3:
      findCar();
        break;
      case 0:
      cout << "Exit" << endl;
        break;
      default:
      cout << "Wrong choice" << endl;
      break;
    }
  } while (choice != 0);
  return 0;
}
```

modular/input.h

```
#ifndef INPUT_H
#define INPUT_H
#include <iostream>
#include <fstream>
#include <cstring>
#include <limits>
#include "car.h"
using namespace std;
void addCar(); // Вводит данные автомобиля с проверками и добавляет запись в файл.
```

```
15  void addCar() {
16    Car car; // Текущая запись автомобиля для ввода, чтения или вывода.
18    cin.ignore(numeric_limits<streamsize>::max(), '\n');
20    cout << "Brand: ";
21    cin.getline(car.brand, 30);
23    while (cin.fail() || strlen(car.brand) == 0) {
24      if (cin.fail()) {
25        cin.clear();
26        cin.ignore(numeric_limits<streamsize>::max(), '\n');
28        cout << "Brand is too long. Maximum 29 characters: ";
29      } else {
30        cout << "Brand cannot be empty" << endl;
31      }
33      cin.getline(car.brand, 30);
34    }
36    cout << "Model: ";
37    cin.getline(car.model, 30);
39    while (cin.fail() || strlen(car.model) == 0) {
40      if (cin.fail()) {
41        cin.clear();
42        cin.ignore(numeric_limits<streamsize>::max(), '\n');
44        cout << "Model is too long. Maximum 29 characters" << endl;
45      } else {
46        cout << "Model cannot be empty" << endl;
47      }
49      cin.getline(car.model, 30);
50    }
52    cout << "Year: ";
54    while (!(cin >> car.year) || car.year < 1886 || car.year > 2026) {
55      cout << "Wrong year. Enter year from 1886 to 2026" << endl;
57      cin.clear();
58      cin.ignore(numeric_limits<streamsize>::max(), '\n');
59    }
61    cout << "Price: ";
63    while (!(cin >> car.price) || car.price <= 0) {
64      cout << "Wrong price. Enter positive number: ";
66      cin.clear();
67      cin.ignore(numeric_limits<streamsize>::max(), '\n');
68    }
70    cin.ignore(numeric_limits<streamsize>::max(), '\n');
72    cout << "Body: ";
73    cin.getline(car.body, 20);
75    while (cin.fail() || strlen(car.body) == 0) {
76      if (cin.fail()) {
77        cin.clear();
78        cin.ignore(numeric_limits<streamsize>::max(), '\n');
80        cout << "Body car is too long. Maximum 19 characters" << endl;
81      } else {
82        cout << "Body car cannot be empty" << endl;
83      }
85      cin.getline(car.body, 20);
86    }
88    cout << "Segment: ";
89    cin.getline(car.segment, 10);
91    while (cin.fail() || strlen(car.segment) == 0) {
92      if (cin.fail()) {
93        cin.clear();
94        cin.ignore(numeric_limits<streamsize>::max(), '\n');
96        cout << "Segment is too long. Maximum 9 characters" << endl;
97      } else {
98        cout << "Segment cannot be empty" << endl;
99      }
101      cin.getline(car.segment, 10);
102    }
104    ofstream file("cars.dat", ios::binary | ios::app); // Поток двоичной записи: новые данные
добавляются в конец файла.
```

```
106    if (!file) {
107      cout << "File open error" << endl;
108    } else {
109      file.write(reinterpret_cast<char*>(&car), sizeof(Car));
110      file.close();
112      cout << "Car added." << endl;
113    }
114  }
116  #endif
```

modular/output.h

```
 1  #ifndef OUTPUT_H
 2  #define OUTPUT_H
 4  #include <iostream>
 5  #include <fstream>
 6  #include <iomanip>
 7  #include <string>
 9  #include "car.h"
11  using namespace std;
13  void showAll(); // Читает файл и выводит все автомобили в виде таблицы.
14  void printHeader(); // Выводит названия столбцов таблицы.
15  void printCar(const Car& car); // Выводит одну запись автомобиля; car передаётся без
копирования и изменения.
17  void showAll() {
18    Car car; // Текущая запись автомобиля для ввода, чтения или вывода.
20    ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла cars.dat.
22    if (!file) {
23      cout << "Error file read" << endl;
24    } else {
25      printHeader();
27      while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
28        printCar(car);
29      }
31      file.close();
32    }
33  }
35  void printHeader() {
36-43    cout << left << setw(31) << "Brand" << setw(31) << "Model" << setw(21) << "Body" <<
setw(11) << "Segment" << setw(8) << "Year" << setw(14) << "Price" << endl;
45    cout << string(116, '-') << endl;
46  }
48  void printCar(const Car& car) {
49-56    cout << left << setw(31) << car.brand << setw(31) << car.model << setw(21) <<
car.body << setw(11) << car.segment << setw(8) << car.year << setw(14) << car.price << endl;
57  }
59  #endif
```

modular/search.h

```
 1  #ifndef SEARCH_H
 2  #define SEARCH_H
 4  #include <iostream>
 5  #include <fstream>
 6  #include <cstring>
 7  #include <limits>
 9  #include "car.h"
10  #include "output.h"
12  using namespace std;
14  void findCar(); // Выводит меню поиска и вызывает поиск по выбранному полю.
15  void searchByBrand(); // Ищет автомобили по подстроке марки с учётом регистра.
16  void searchByModel(); // Ищет автомобили по подстроке модели с учётом регистра.
17  void searchByYear(); // Ищет автомобили в заданном диапазоне годов, включая границы.
18  void searchByPrice(); // Ищет автомобили в заданном диапазоне цен, включая границы.
19  void searchByBody(); // Ищет автомобили по подстроке типа кузова с учётом регистра.
20  void searchBySegment(); // Ищет автомобили по подстроке сегмента с учётом регистра.
22  void findCar() {
23    int searchChoice; // Поле поиска: 1–6; значение 0 возвращает в основное меню.
25    cout << "search by: " << endl;
```

```
26    cout << "1 - brand" << endl;
27    cout << "2 - model" << endl;
28    cout << "3 - year" << endl;
29    cout << "4 - price" << endl;
30    cout << "5 - body" << endl;
31    cout << "6 - segment " << endl;
32    cout << "0 - back" << endl;
34    while (!(cin >> searchChoice) || searchChoice < 0 || searchChoice > 6) {
35      cout << "Wrong choice. Enter 0-6: ";
37      cin.clear();
38      cin.ignore(numeric_limits<streamsize>::max(), '\n');
39    }
41    switch (searchChoice) {
42    case 1:
43      searchByBrand();
44      break;
46    case 2:
47      searchByModel();
48      break;
50    case 3:
51      searchByYear();
52      break;
54    case 4:
55      searchByPrice();
56      break;
58    case 5:
59      searchByBody();
60      break;
62    case 6:
63      searchBySegment();
64      break;
66    case 0:
67      break;
69    default:
70      break;
71    }
72  }
74  void searchByBrand() {
75    Car car; // Текущая запись автомобиля для ввода, чтения или вывода.
76    char searchBrand[30]; // Подстрока для поиска по марке.
78    cin.ignore(numeric_limits<streamsize>::max(), '\n');
80    cout << "Enter Brand: ";
81    cin.getline(searchBrand, 30);
83    while (cin.fail() || strlen(searchBrand) == 0) {
84      if (cin.fail()) {
85        cin.clear();
86        cin.ignore(numeric_limits<streamsize>::max(), '\n');
88        cout << "Brand is too long. Maximum 29 characters" << endl;
89      } else {
90        cout << "Brand cannot be empty" << endl;
91      }
93      cin.getline(searchBrand, 30);
94    }
96    ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла cars.dat.
98    if (!file) {
99      cout << "File open error" << endl;
100   } else {
101     printHeader();
103     while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
104       if (strstr(car.brand, searchBrand) != nullptr) {
105         printCar(car);
106       }
107     }
109     file.close();
110   }
111 }
```

```
113 void searchByModel() {
114   Car car; // Текущая запись автомобиля для ввода, чтения или вывода.
115   char searchModel[30]; // Подстрока для поиска по модели.
117   cin.ignore(numeric_limits<streamsize>::max(), '\n');
119   cout << "Enter model: " << endl;
120   cin.getline(searchModel, 30);
122   while (cin.fail() || strlen(searchModel) == 0) {
123     if (cin.fail()) {
124       cin.clear();
125       cin.ignore(numeric_limits<streamsize>::max(), '\n');
127       cout << "Model is too long. Maximum characters 29" << endl;
128     } else {
129       cout << "Model cannot be empty" << endl;
130     }
132     cin.getline(searchModel, 30);
133   }
135   ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла cars.dat.
137   if (!file) {
138     cout << "File open error" << endl;
139   } else {
140     printHeader();
142     while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
143       if (strstr(car.model, searchModel) != nullptr) {
144         printCar(car);
145       }
146     }
148     file.close();
149   }
150 }
152 void searchByYear() {
153   Car car; // Текущая запись автомобиля для ввода, чтения или вывода.
154   int minYear; // Нижняя граница диапазона годов.
155   int maxYear; // Верхняя граница диапазона годов; не меньше нижней.
157   cout << "From year: " << endl;
159   while (!(cin >> minYear) || minYear < 1886 || minYear > 2026) {
160     cout << "Wrong year. Enter year from 1886 to 2026" << endl;
162     cin.clear();
163     cin.ignore(numeric_limits<streamsize>::max(), '\n');
164   }
166   cout << "To year: " << endl;
168-169   while (!(cin >> maxYear) || maxYear < 1886 || maxYear > 2026 || maxYear < minYear) {
170     cout << "Wrong year. Enter year from " << minYear << " to 2026: ";
172     cin.clear();
173     cin.ignore(numeric_limits<streamsize>::max(), '\n');
174   }
176   ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла cars.dat.
178   if (!file) {
179     cout << "File open error" << endl;
180   } else {
181     printHeader();
183     while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
184       if (car.year >= minYear && car.year <= maxYear) {
185         printCar(car);
186       }
187     }
189     file.close();
190   }
191 }
193 void searchByPrice() {
194   Car car; // Текущая запись автомобиля для ввода, чтения или вывода.
195   int minPrice; // Нижняя граница диапазона цен.
196   int maxPrice; // Верхняя граница диапазона цен; не меньше нижней.
198   cout << "From price" << endl;
200   while (!(cin >> minPrice) || minPrice < 1) {
201     cout << "Wrong price. Enter correct price." << endl;
203     cin.clear();
```

```
204     cin.ignore(numeric_limits<streamsize>::max(), '\n');
205   }
207   cout << "To price" << endl;
209   while (!(cin >> maxPrice) || maxPrice < 1 || maxPrice < minPrice) {
210     cout << "Wrong price. Enter correct price." << endl;
212     cin.clear();
213     cin.ignore(numeric_limits<streamsize>::max(), '\n');
214   }
216   ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла cars.dat.
218   if (!file) {
219     cout << "File open error" << endl;
220   } else {
221     printHeader();
223     while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
224       if (car.price >= minPrice && car.price <= maxPrice) {
225         printCar(car);
226       }
227     }
229     file.close();
230   }
231 }
233 void searchByBody() {
234   Car car; // Текущая запись автомобиля для ввода, чтения или вывода.
235   char searchBody[20]; // Подстрока для поиска по типу кузова.
237   cin.ignore(numeric_limits<streamsize>::max(), '\n');
239   cout << "Enter body: " << endl;
240   cin.getline(searchBody, 20);
242   while (cin.fail() || strlen(searchBody) == 0) {
243     if (cin.fail()) {
244       cin.clear();
245       cin.ignore(numeric_limits<streamsize>::max(), '\n');
247       cout << "Body is too long. Maximum characters 19" << endl;
248     } else {
249       cout << "Body cannot be empty" << endl;
250     }
252     cin.getline(searchBody, 20);
253   }
255   ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла cars.dat.
257   if (!file) {
258     cout << "File open error" << endl;
259   } else {
260     printHeader();
262     while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
263       if (strstr(car.body, searchBody) != nullptr) {
264         printCar(car);
265       }
266     }
268     file.close();
269   }
270 }
272 void searchBySegment() {
273   Car car; // Текущая запись автомобиля для ввода, чтения или вывода.
274   char searchSegment[10]; // Подстрока для поиска по сегменту.
276   cin.ignore(numeric_limits<streamsize>::max(), '\n');
278   cout << "Enter segment: " << endl;
279   cin.getline(searchSegment, 10);
281   while (cin.fail() || strlen(searchSegment) == 0) {
282     if (cin.fail()) {
283       cin.clear();
284       cin.ignore(numeric_limits<streamsize>::max(), '\n');
286       cout << "Segment is too long. Maximum characters 9" << endl;
287     } else {
288       cout << "Segment cannot be empty" << endl;
289     }
291     cin.getline(searchSegment, 10);
292   }
```

```
294   ifstream file("cars.dat", ios::binary); // Поток двоичного чтения записей из файла cars.dat.
296   if (!file) {
297     cout << "File open error" << endl;
298   } else {
299     printHeader();
301     while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
302       if (strstr(car.segment, searchSegment) != nullptr) {
303         printCar(car);
304       }
305     }
307     file.close();
308   }
309 }
311 #endif
```